

SEMANA 10

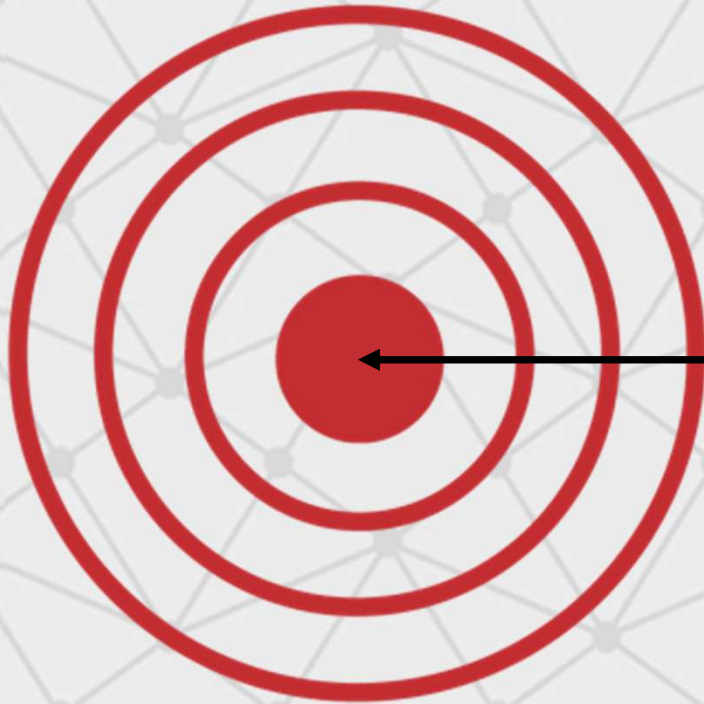


Curso: Programación
Concurrente y Distribuida

Docente: Ms. Carlos Jara
García



Logro de sesión



- Al finalizar la sesión, el estudiante **construye servicios distribuidos.**



Sesión 10: Servicios Distribuidos

Contenido:

- Topología de red
- Sockets TCP/IP
- Web Server http

Ideas



¿Qué entendemos por servicios distribuidos?

Topología de red



Figure 1-1: A direct connection between two nodes

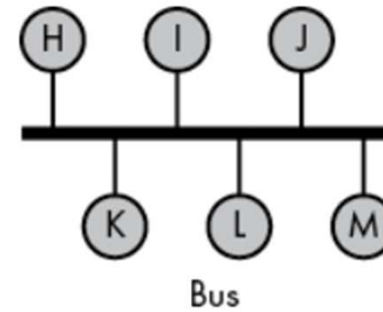


Figure 1-3: Nodes connected in a bus topology

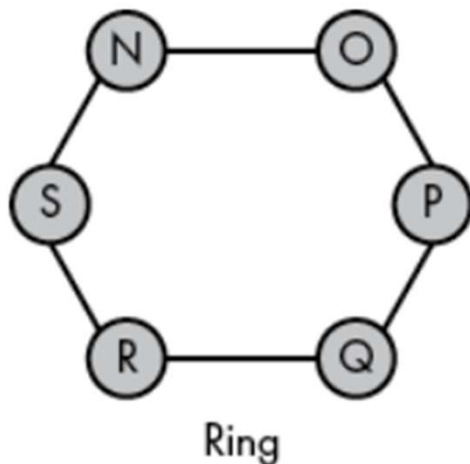


Figure 1-4: Nodes arranged in a ring, with traffic traveling in a single direction

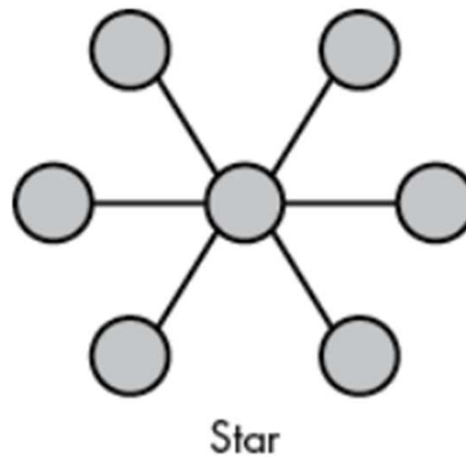


Figure 1-5: Nodes connected to a central node, which handles traffic between nodes

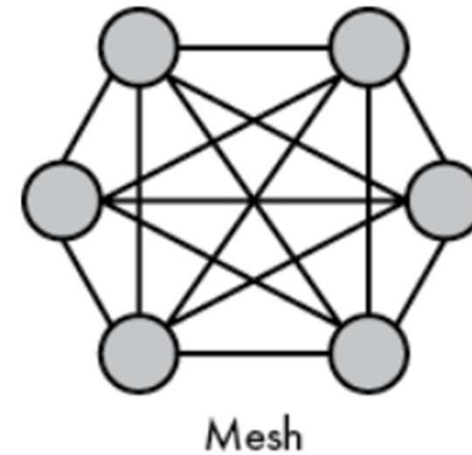


Figure 1-6: Interconnected nodes in a mesh network

Sockets TCP/IP



```
// 10.21.61.171
func main() {
    con, _ := net.Dial("tcp", "10.21.61.155:8000")
    defer con.Close()
    fmt.Fprintln(con, "jalados!")
}
```

Rol Cliente

```
func main() {
    ln, _ := net.Listen("tcp", "10.21.61.155:8000")
    defer ln.Close()
    con, _ := ln.Accept()
    defer con.Close()
    r := bufio.NewReader(con)
    msg, _ := r.ReadString('\n')
    fmt.Print(msg)
}
```

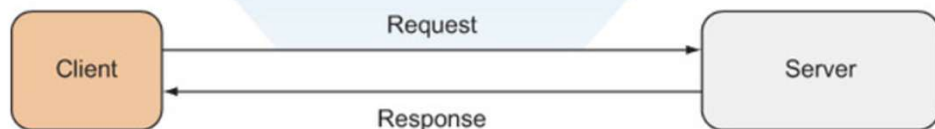
Rol Servidor

Web Server http

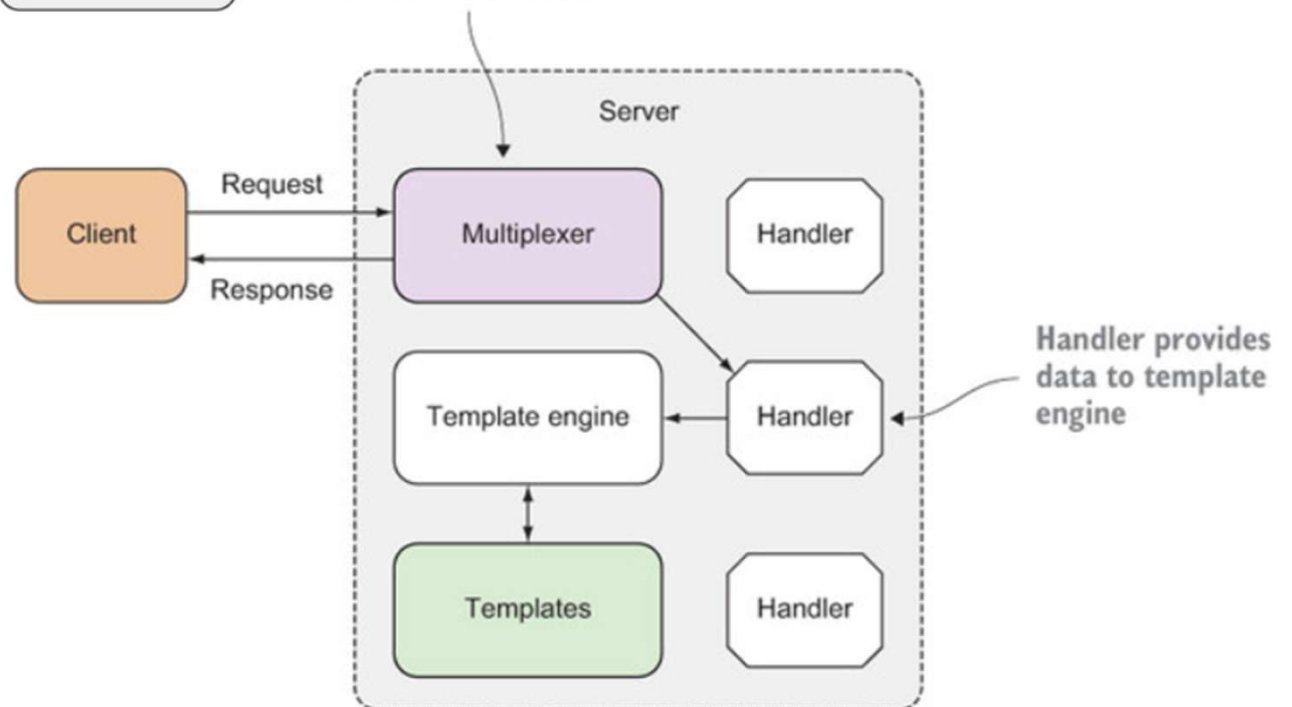


Format of request is suggested by the web app,
in hyperlinks on HTML pages provided to client by server.

http://<servername>/<handlername>?<parameters>



Multiplexer inspects
URL request, redirects
to correct handler

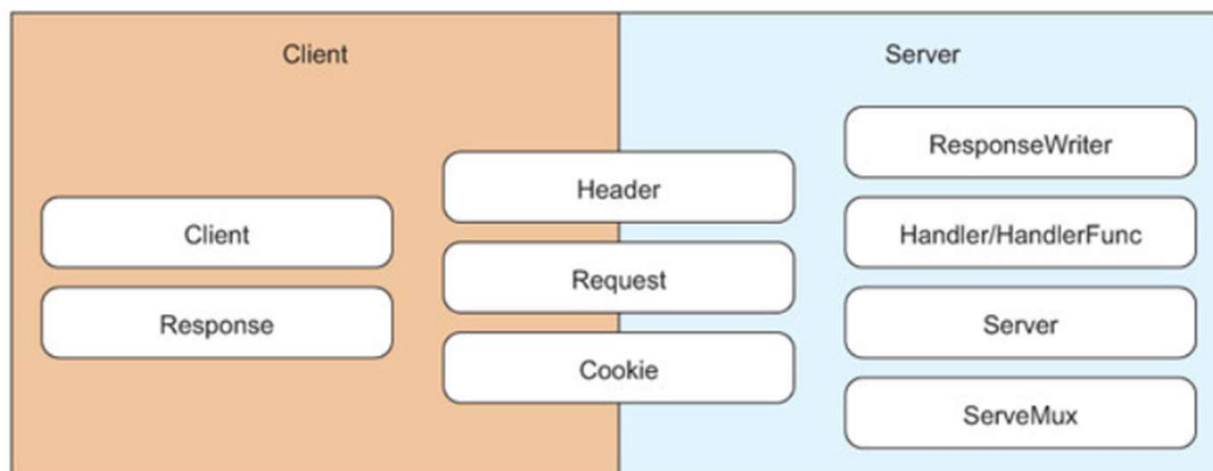


Web Server http



The net/http library is divided into two parts, with various structs and functions supporting either one or both (see [figure 3.1](#)):

Figure 3.1. Chaining handlers



- **Client** —Client, Response, Header, Request, Cookie
- **Server** —Server, ServeMux, Handler/HandleFunc, ResponseWriter, Header, Request, Cookie

Web Server http



```
package main

import (
    "net/http"
)

func main() {
    http.ListenAndServe("", nil)
}
```

Simple Web Server

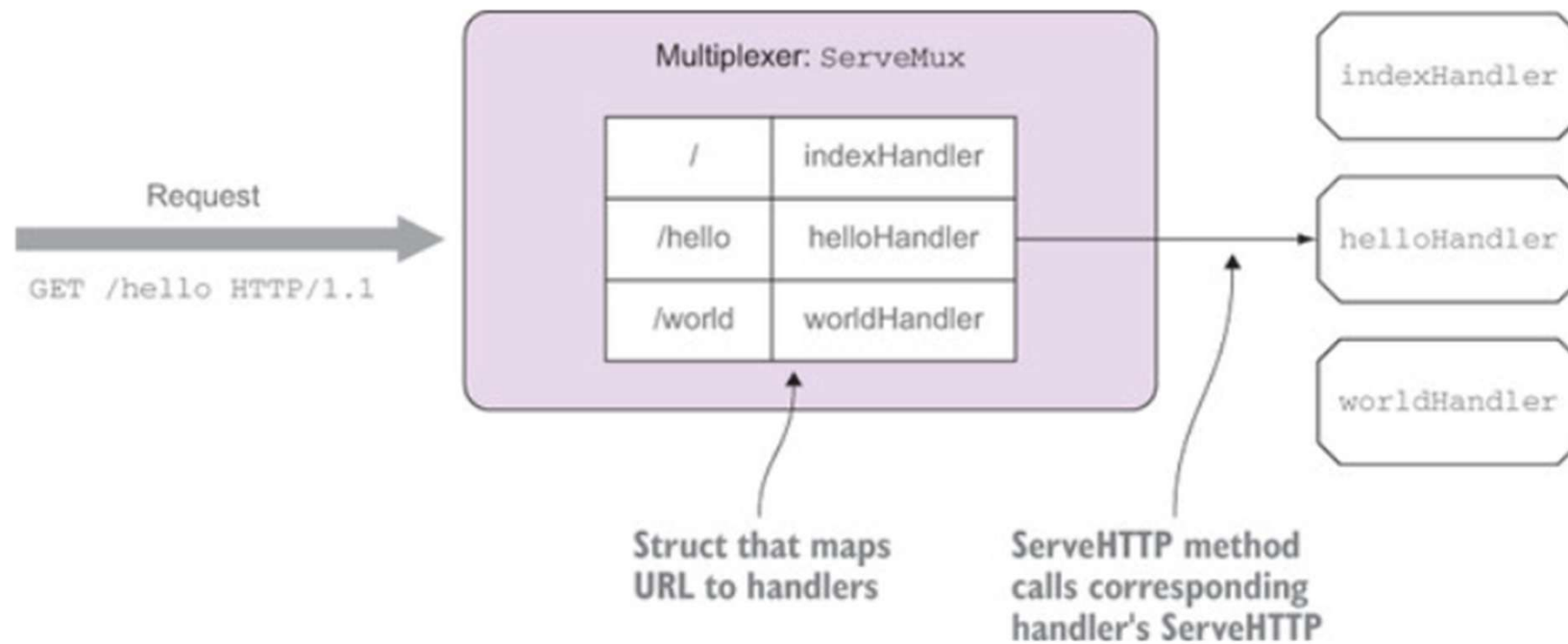
```
package main

import (
    "net/http"
)

func main() {
    server := http.Server{
        Addr:    "127.0.0.1:8080",
        Handler: nil,
    }
    server.ListenAndServe()
}
```

Web Server con configuración adicional

Web Server http



Multiplexación de solicitudes a controladores

Web Server http



```
package main

import (
    "fmt"
    "net/http"
)

func hello(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Hello!")
}

func world(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "World!")
}

func main() {
    server := http.Server{
        Addr: "127.0.0.1:8080",
    }
    http.HandleFunc("/hello", hello)
    http.HandleFunc("/world", world)

    server.ListenAndServe()
}
```

Practicamos



Manos a la Obra.... Programemos un socket y un web server



Gracias